SLOWLY CHANGING DIMENSION (SCD)

Data Engineering - Quick Revision Notes

1 What is SCD?

Slowly Changing Dimension (SCD) is a data-warehouse technique used to manage changes in dimension data over time. Dimensions describe customers, products, employees, stores, or locations.

Example: A customer moves from Hyderabad to Bengaluru. SCD determines whether to overwrite the old city, preserve both versions, or keep limited history.

Key benefit
SCD helps answer "What is true now?" and, when history is retained, "What was true at that time?"

Term	Simple meaning	Example
Dimension table	Descriptive details used to analyze facts.	Customer, product, store
Current value	Latest known value.	City = Bengaluru
Historical value	A previous value kept for analysis.	City used to be Hyderabad
Business key	Identifier from source/business system.	`customer_id = 101`
Surrogate key	Warehouse-generated key for a dimension row.	`customer_sk = 5001`

2 Why do we use SCD?

Advantage	Explanation
Historical analysis	Compare results using old and new customer, product, or location details.
Accurate reporting	Use latest values or values that were valid when an event occurred.
Auditability	History-preserving types help show when a value changed.
Business decisions	Understand how changes in customer segment, address, or category affect trends.

Remember: SCD manages changes in **dimension attributes**, not measures such as sales amount.

SCD TYPES

Ask: how much history should be kept?

3 SCD Type 0 — Retain the Original

The original value is kept and changes are generally not applied. Useful for attributes that should remain fixed, such as an original registration date.

Before	Incoming value	Stored result
Registration date: 2022-01-10	2022-02-10	2022-01-10 (unchanged)

4 SCD Type 1 — Overwrite (No History)

The old value is overwritten with the new value. Only the latest value remains; the previous value is not preserved in that row.

Before update	New value	After update
`customer_id 101 | city = Hyderabad`	`city = Bengaluru`	`customer_id 101 | city = Bengaluru`

Use when: correcting spelling or when history is not required.

Limitation: the previous city cannot be recovered from this dimension row.

5 SCD Type 2 — Add a New Row (Full History)

A new row is inserted for every tracked change. The old row is retained, commonly with a surrogate key and effective-date/current-row columns.

customer_sk	customer_id	city	valid_from	valid_to	is_current
5001	101	Hyderabad	2023-01-01	2024-03-14	`FALSE`
5088	101	Bengaluru	2024-03-15	9999-12-31	`TRUE`

The business key stays the same, but each historical version gets a different surrogate key.

Use when: historical reporting matters, such as sales by a customer's region at the time of purchase.

SCD TYPES • COMPARISON & IMPLEMENTATION

Quick revision for interviews and data-engineering projects

6 SCD Type 3 — Previous-Value Column

A new column stores a previous value alongside the current value. This usually preserves only limited history.

customer_id	previous_city	current_city
101	Hyderabad	Bengaluru

Use when: the business needs a current-versus-previous comparison.

Limitation: multiple older changes are not fully retained in this simple design.

7 Other SCD Types

Type	Approach	History kept?
Type 4	Keep current dimension data in one table and history in a separate history table.	Yes, in separate table
Type 6	Hybrid approach combining Type 1, Type 2, and Type 3 ideas.	Yes, hybrid history

Most introductory interviews focus on Types 1, 2, and 3. Types 0, 4, and 6 may also appear depending on the project.

8 SCD Types at a Glance

Type	What happens?	History	Memory trick
0	Keep original value	Original only	Freeze it
1	Overwrite old value	No history	Replace it
2	Insert a new row	Full row history	New row
3	Add previous-value column	Limited history	New column
4	Separate history table	History table	Split tables
6	Hybrid of 1 + 2 + 3	Hybrid history	Mix types

9 Type 2 Processing — Basic Workflow

1. Read
Load incoming records

2. Compare
Match business key and tracked attributes

3. Expire
Close old current row

4. Date it
Set end date/current flag

5. Insert
Add new version and key

6. Validate
One current row per key

10 Simple SQL Example — Type 2

```
UPDATE dim_customer SET valid_to = DATE '2024-03-14', is_current = FALSE WHERE customer_id = 101 AND is_current = TRUE; -- Then insert the new version INSERT INTO dim_customer (customer_sk, customer_id, city, valid_from, valid_to, is_current) VALUES (5088, 101, 'Bengaluru', DATE '2024-03-15', DATE '9999-12-31', TRUE);
```

Illustrative SQL pattern: generate surrogate keys safely, follow the project's date convention, and handle duplicate or replayed source records.

Interview tip
Complete history? **Type 2**. Overwrite old value? **Type 1**. Previous value in another column? **Type 3**.